QUESTION 3

Title:

National Census Data Sorting System Using Merge Sort

Aim:

To analyze the application of Merge Sort in sorting large-scale census records based on region codes and demographic categories, and to evaluate its stability, scalability, and performance.

Problem Understanding:

A national census platform stores millions of citizen records containing region codes and demographic information. Efficient sorting of these records is essential for data analysis, report generation, and decision-making.

Merge Sort is a divide-and-conquer algorithm that recursively divides data into smaller subarrays, sorts them, and merges them back together. It provides stable sorting and predictable performance, making it suitable for handling large structured datasets.

Algorithm:

- Divide the list of records into two halves.
- Recursively divide each half until one element remains.
- Sort and merge the smaller subarrays.
- Continue merging until the complete dataset is sorted.
- Display the sorted records.

Pseudocode:

```
MERGE_SORT(arr)

IF length(arr) <= 1
RETURN arr

mid = length(arr)/2
```

left = MERGE_SORT(left half)
right = MERGE_SORT(right half)

MERGE(left,right)

RETURN merged array

Code:

```
import time

def merge_sort(arr):

if len(arr) <= 1:
    return arr
mid = len(arr) // 2

left = merge_sort(arr[:mid])
right = merge_sort(arr[mid:])

result = []

i = j = 0

while i < len(left) and j < len(right):

    if left[i] < right[j]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1

result.extend(left[i:])
result.extend(right[j:])

return result

data = [38, 27, 43, 3, 9, 82, 10]

start = time.time()

sorted_data = merge_sort(data)

end = time.time()

print("Original Data:", data)
print("Sorted Data:", sorted_data)
print("Execution Time:", round(end-start,6), "seconds")
```

Sample Output:

Original Data: [38, 27, 43, 3, 9, 82, 10]

Sorted Data: [3, 9, 10, 27, 38, 43, 82]

Execution Time: 0.00001 seconds

Analysis:

Divide and Conquer Principle:

Divide:
The dataset is repeatedly divided into two halves.

Example:

[38,27,43,3,9,82,10]

↓

[38,27,43] [3,9,82,10]

Conquer:
Each half is recursively sorted.

Combine:
Sorted halves are merged to produce the final sorted list.

Stability:
Merge Sort is stable. Records having the same region code maintain their original order.

Scalability:
Merge Sort performs efficiently even for millions of records and is suitable for large-scale census applications.

Predictable Performance:
Merge Sort consistently performs O(n log n) operations in all cases.

Time Complexity:

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n log n)

Space Complexity:

O(n)

Additional memory is required during the merging process.

Result:

The experiment shows that Merge Sort is highly suitable for national census data sorting because it provides stable sorting, predictable O(n log n) performance, and excellent scalability for handling large structured datasets. Its divide-and-conquer approach makes it an efficient solution for large-scale government data processing systems.